

Reviewer Pack — Brooks Quasimorphism Magnitude Lower-Bounds Formula Depth vi...

Entry 5013aa48b582 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-27 09:53:25 UTC

Brooks Quasimorphism Magnitude Lower-Bounds Formula Depth via Barrington Lifts

Entry ID: 5013aa48b582 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-27 09:53:25 UTC

1. Conjecture

Field A (mathematical branch): Geometric group theory: Brooks counting quasimorphisms on the free group F_2 (bounded cohomology $H_b^2(F_2; \mathbb{R})$, Bestvina–Fujiwara) **Field B** (complexity object): NC^1 / Boolean formula depth via Barrington’s width-5 S_5 branching program lifted to F_2

Statement:

For a Boolean formula φ of syntactic depth d over n variables, build Barrington’s canonical width-5 ground BP B_φ (length 4^d) and lift each S_5 factor to $F_2 = \langle a, b \rangle$ via the surjection $a \mapsto (1\ 2\ 3\ 4\ 5)$, $b \mapsto (1\ 2)$, giving a word $w_\varphi(x) \in F_2$ of length 4^d for every input $x \in \{0, 1\}^n$. Let $\tau(w) := \#\{\text{occurrences of the reduced subword } aba \text{ in } w\} - \#\{\text{occurrences of } a^{-1}b^{-1}a^{-1} \text{ in } w\}$ be the Brooks counting quasimorphism on F_2 with seed aba . Conjecture: there exist absolute constants $c_1, c_2 > 0$ such that for every φ with $d \geq 2$, $c_1 \cdot 2^d \leq \max_{x \in \{0, 1\}^n} |\tau(w_\varphi(x))| \leq c_2 \cdot 2^d$, so $\text{depth}(\varphi) = \log_2 \max_x |\tau(w_\varphi(x))| \pm O(1)$.

Rationale (proposer’s reasoning):

Barrington’s recursion nests commutators d -deep, and in the free group F_2 nested commutators are precisely the words on which Brooks quasimorphisms grow linearly per nesting level (Bavard duality, scl-detection). Solvable groups have bounded commutator length so this signal collapses, but F_2 — the universal Barrington target — preserves it, suggesting a single F_2 -word invariant can certify NC^1 depth orthogonally to communication, polynomial-method, or approximation-based lower bounds. No prior work I know of pairs Brooks quasimorphisms with Barrington BPs.

Taxonomy category: BARRINGTON_ALG (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): eb498c41689028a5

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across ≥ 30 random + structured formulas per depth $d \in \{2, 3, 4, 5\}$ on $n \in \{4, 6, 8\}$ vars, compute $r(\varphi) = \max_x |\tau(w_\varphi(x))|/2^d$. SUPPORTED iff median r per depth $\in [0.25, 4]$, medians monotone non-decreasing in d , and no $d \geq 3$ formula yields $r < 0.1$ or $r > 10$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.90	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.78	SAFE	SAFE
ALGEBRIZATION	SAFE	0.83	SAFE	SAFE
KARP_LIPTON	SAFE	0.90	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Brooks quasimorphism Barrington branching program free group - bounded cohomology free group formula depth lower bound NC1 - counting quasimorphism word length permutation branching program S_5

Top relevant hits considered: - [http://arxiv.org/abs/1911.10855v2] $\hat{G}$ -invariant quasimorphisms and symplectic geometry of surfaces - [http://arxiv.org/abs/hep-ph/0610012v1] Tevatron-for-LHC Report of the QCD Working Group - [http://arxiv.org/abs/1710.02392v2] The cup product of Brooks quasimorphisms - [http://arxiv.org/abs/1901.01301v1] Second bounded cohomology and WWPD - [http://arxiv.org/abs/math/0701214v2] Graphs, free groups and the Hanna Neumann conjecture - [http://arxiv.org/abs/1007.1875v2] Lower Bounds for Quantum Oblivious Transfer - [http://arxiv.org/abs/1512.02171v5] Right-jumps and pattern avoiding permutations - [http://arxiv.org/abs/1001.4392v1] Counting Finite Languages by Total Word Length

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import json
from collections import defaultdict

def barrington_construction(formula):
    stack = [[]]
    for node in formula:
        if node[0] == 'AND':
            stack.append([])
        elif node[0] == 'OR':
            stack[-1].extend(stack.pop())
        elif node[0] == 'NOT':
            stack[-1][len(stack[-1]) - 1] = (stack[-1][len(stack[-1]) - 1][0], '1', '0')
        else:
            stack[-1].append((len(stack[-1]) + 1, '0', '1'))
    return stack[0]
```

```

def lift_to_f2(triples):
    a = [(1, 2, 3, 4, 5)]
    b = [(1, 2)]
    f2_word = []
    for triple in triples:
        var_index, sigma_true, sigma_false = triple
        if sigma_true == '0':
            f2_word.extend(a)
        elif sigma_true == '1':
            f2_word.extend(b)
        if sigma_false == '0':
            f2_word.extend(b)
        elif sigma_false == '1':
            f2_word.extend(a)
    return reduce_f2_word(f2_word)

def reduce_f2_word(word):
    stack = []
    for letter in word:
        if stack and (stack[-1] == ('a', 'b') or stack[-1] == ('b', 'a')):
            stack.pop()
        else:
            stack.append(letter)
    return stack

def count_aba(word):
    aba_count = 0
    a_inv_b_inv_a_inv_count = 0
    for i in range(len(word) - 2):
        if word[i:i+3] == ('a', 'b', 'a'):
            aba_count += 1
        elif word[i:i+4] == ('a', 'b', 'a', 'b') or word[i:i+4] == ('b', 'a', 'b', 'a'):
            a_inv_b_inv_a_inv_count += 1
    return aba_count - a_inv_b_inv_a_inv_count

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_formula(n, depth):
        if depth == 0:
            return [('NOT', random.choice([0, 1]))]
        else:
            op = random.choice(['AND', 'OR'])
            left = generate_formula(n // 2, depth - 1)
            right = generate_formula(n // 2, depth - 1)
            if op == 'AND':
                return [('AND', left, right)]
            elif op == 'OR':
                return [('OR', left, right)]

    def evaluate_formula(formula, x):
        stack = []
        for node in formula:
            if node[0] == 'NOT':
                stack.append(not evaluate_formula(node[1], x))
            else:

```

```

        a, b = x[node[2]], x[node[3]]
        if node[0] == 'AND':
            stack.append(a and b)
        elif node[0] == 'OR':
            stack.append(a or b)
    return stack[-1]

n_values = [4, 6, 8]
depth_values = [2, 3, 4, 5]
results = defaultdict(list)

for n in n_values:
    for d in depth_values:
        formula = generate_formula(n, d)
        max_tau = 0
        for _ in range(10):
            x = {i: random.choice([0, 1]) for i in range(n)}
            triples = barrington_construction(formula)
            f2_word = lift_to_f2(triples)
            tau = count_abn(f2_word)
            max_tau = max(max_tau, abs(tau))
        results[(n, d)].append(max_tau)

metric_values = [max(results[(n, d)] for n in n_values) / (2 ** d) for d in depth_values]
mean_value = sum(metric_values) / len(depth_values)
std_value = (sum((x - mean_value) ** 2 for x in metric_values) / len(depth_values)) ** 0.5
support_fraction = sum(1 for v in metric_values if 0.25 <= v <= 4) / len(depth_values)

conjecture_holds = support_fraction >= 0.8 and all(metric_values[i] <= metric_values[i + 1] for i in range(len(metric_values) - 1))
counterexample = "" if conjecture_holds else f"median ratio {mean_value} not in [0.25,4] or no"

return {
    "metric_name": "max_tau_over_2^d",
    "metric_value": mean_value,
    "instances_tested": len(depth_values),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [11, 23, 37, 53, 71]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {json.dumps(result)}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = (sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results)) ** 0.5
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])

```

```

    print(f"RESULT: FALSIFIED counterexample=\"median ratio {mean_value} not in [0.25,4] or no
else:
    print("RESULT: INCONCLUSIVE")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_252ab5e3.py", line 130, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_252ab5e3.py", line 103, in run_trial
    triples = barrington_construction(formula)
              ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_252ab5e3.py", line 28, in barrington_co
    return stack[0]
           ~~~~~^
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with an IndexError in barrington_construction before any data was collected, so neither the support nor the falsification condition can be evaluated. | next: Fix the Barrington construction (likely a stack underflow when handling leaf/literal gates or odd recursion base cases) and rerun the pre-registered protocol across $d \in \{2,3,4,5\}$.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	133581
2	preregistration	claude_max	opus	0	0	7305
3	novelty	claude_max	opus	0	0	4377
4	novelty	claude_max	opus	0	0	7882
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18625
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15819
7	judge	claude_max	opus	0	0	4744

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 192334 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in research/audit/5013aa48b582.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/5013aa48b582.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/5013aa48b582.tar.gz (if generated)